	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 5

Lab Title: **Doubly Circular Linked lists and stack**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective: Implementation of Doubly circular linked lists and stackin C++ l

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

LAB REPORT : 5

Tittle : Stack and doubly circular

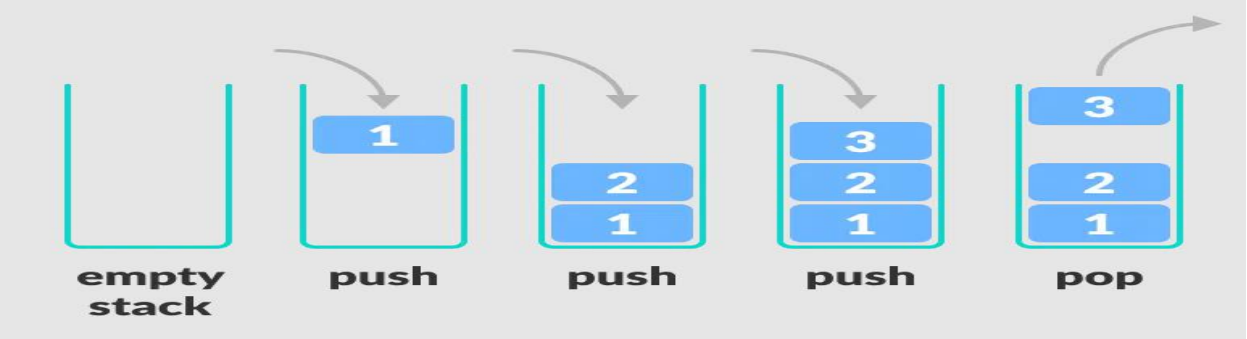
Name : Muhamamd waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 24 th October 24

Stack:



Circular Linked lists



Lab Tasks

1. Make a circular linked list with following functions:

- **Insert Link function**
- **Display function**
- **Delete Node function**

In data section add student name, roll number and department information. Your program must be generic, i.e. program should not close before the user doesn't allow to exit.

Make a function in above program to search student by its: □ Roll Number □ Name.

Code:

```
#include<iostream>
#include<string>
#include<sstream>
using namespace std;
class node{
public:
string name;
int id;
string dep;
node* next;
node* prev;
node(string,int,string);};
node::node(string n,int i,string d){
name=n;
id=i;
dep=d;
next=NULL;
prev=NULL;}
class linklist{
private:
node* head;
node* tail;
public:
linklist();
int length();
void insert_node(string,int ,string);
void display_nodes();
void search();//search by name or id;
void delete_node();};
void linklist::delete_node(){
int pos;
int len=length();
cout<<"Total nodes: "<<length()<<endl;
cout<<"Enter position: ";cin>>pos;
if(head==NULL){
cout<<"List is empty:\n";
return;}
if(pos<=0 || pos>len){
cout<<"Invalid Position:\n";
return;}
else if(pos==1){
node* temp=head;
tail->next=head->next;
head->next->prev=tail;
head=head->next;
delete temp;
cout<<"Head deleted Successfully:\n";
return;}
else if(pos==len){
node* del=tail;
tail->prev->next=head;
head->prev=tail->prev;
tail=tail->prev;
delete del;
cout<<"Tail deleted successfully:\n";
return;}
else{
node* temp=head;
for(int i=1;i<pos-1;i++){
temp=temp->next;}
node* del=temp->next;
temp->next=del->next;
del->next->prev=temp;
delete del;
```

```
 97 switch(c1){
 98 case 1:{
 99 node* last=head;
100 cin.ignore(200,'\n');
101 do{
102 int o=++c;
103 cout<<"===(Student "<<o<<")===\n"
104 cout<<"Name: "<<last->name<<endl;
105 cout<<"ID: "<<last->id<<endl;
106 cout<<"Department:"<<last->dep<<e
107 last=last->next;
108 }while(last!=head);
109 break;}
110 case 2:{
111 node* last=tail;
112 cin.ignore(200,'\n');
113 do{
114 int r=++c;
115 cout<<"===(Student "<<r<<")===\n"
116 cout<<"Name: "<<last->name<<endl;
117 cout<<"ID: "<<last->dep<<endl;
118 last=last->prev;
119 }while(last!=tail);
120 break;}}}
121 int linklist::length(){
122 if(head==NULL) return 0;
123 int count=0;
124 node* temp=head;
125 do{
126 ++count;
127 temp=temp->next;
128 }while(temp!=head);
```

```
129 return count;}
130 void linklist::insert_node(string n,int i,strin
131 node* newnode=new node(n,i,d);
132 cout<<"1: Insert at head:\n";
133 cout<<"2: Insert at tail:\n";
134 cout<<"3:Insert at specific position:\n";
135 int c;cout<<"Enter choice:";cin>>c;
136 node* temp=head;
137 if(head==NULL){
138 head=tail=newnode;
139 head->next=head;
140 head->prev=head;
141 cout<<"Node added as Head(-<First Node>-):\n";
142 return;}
143 switch(c){
144 case 1:{
145 newnode->next=head;
146 newnode->prev=tail;
147 head->prev=newnode;
148 tail->next=newnode;
149 head=newnode;
150 cout<<"Node added as Head Successfully :\n";
151 break;}
152 case 2:{
153 newnode->next=head;
154 tail->next=newnode;
155 head->prev=newnode;
156 newnode->prev=tail;
157 tail=newnode;
158 cout<<"Node added as Tail successfully:\n";
159 break;}
160 case 3:{
```

```
160 case 3:{
161 int pos;
162 int len=length();
163 cout<<"Enter position:";
164 cin>>pos;
165 if(head==NULL){
166 head=tail=newnode;
167 head->next=head;
168 head->prev=head;
169 cout<<"Node added as head(<First Nc
170 return;}
171 else if(pos<=0 || pos>len){
172 cout<<"Invalid psoition:";}
173 else if(pos==1){
174 newnode->next=head;
175 newnode->prev=tail;
176 head->prev=newnode;
177 tail->next=newnode;
178 head=newnode;
179 cout<<"Node added as Head successfu
180 return;}
181 else if(pos==len){
182 newnode->next=head;
183 tail->next=newnode;
184 head->prev=newnode;
185 newnode->prev=tail;
186 tail=newnode;
187 cout<<"Node added as Tail:\n";
188 return;}
189 else{
190 for(int i=1;i<pos-1;i++){
191 temp=temp->next;}
```

```
192 newnode->next=temp->next;
193 newnode->prev=temp;
194 temp->next=newnode;
195 temp->next->prev=newnode;
196 cout<<"Node deleted";
197 cout<<" successfuly at given  position "<
198 return;}
199 break;}}}
200 linklist::linklist(){
201 head=NULL;
202 tail=NULL;}
203 int main(){
204 linklist l1;string n,d;
205 while(1){
206 system("cls");
207 cout<<"========(Main Menu)========\n";
208 cout<<"1: Add New Student:\n";
209 cout<<"2: Display Students:\n";
210 cout<<"3: Search Student:\n";
211 cout<<"4: Delete Student(Give position of
212 cout<<"5: Exit the program:\n:";
213 int c,i;
214 cout<<"Enter Choice:";cin>>c;
215 switch(c){
216 case 1:{
217 cin.ignore(200,'\n');
218 cout<<"Enter student name: "; getline(cin
219 cout<<"Enter student ID: ";cin>>i;
220 cin.ignore(200,'\n');
221 cout<<"Enter student Department: ";getlin
222 cout<<endl;
223 l1.insert_node(n,i,d);
```

```
224     system("pause");
225     break;}
226     case 2:{
227     l1.display_nodes();
228     system("pause");
229     break;}
230     case 3:{
231     //Search by name or id logic(used);
232     l1.search();
233     system("pause");
234     break;}
235     case 4:{
236     l1.delete_node();
237     system("pause");
238     break;}
239     case 5:{
240     cout<<"=======END=========\n";
241     exit(1);
242     break;}
243     default:{
244     cout<<"Invalid chocie:\n";
245     system("pause");}}}
246     return 0;}
```

OUTPUT:

```
========(Main Menu)========
1: Add New Student:
2: Display Students:
3: Search Student:
4: Delete Student(Give position of node:)
5: Exit the program:
:Enter Choice:1
Enter student name: Muhamamd Waqas
Enter student ID: 240756
Enter student Department: Computer engineering

1: Insert at head:
2: Insert at tail:
3:Insert at specific position:
Enter choice:2
Node added as Head(-<First Node>-):
Press any key to continue . . .
```

C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe

```
========(Main Menu)========
1: Add New Student:
2: Display Students:
3: Search Student:
4: Delete Student(Give position of node:)
5: Exit the program:
:Enter Choice:2
1: Display from head to tail:
2: display fro tail to head:
Enter chocie:1
===(Student 1)===
Name: Muhamamd Waqas
ID: 240756
Department:Computer engineering
===(Student 2)===
Name: Ali
ID: 112233
Department:Ai engineering
Press any key to continue . . .
```

C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe

```
========(Main Menu)========
1: Add New Student:
2: Display Students:
3: Search Student:
4: Delete Student(Give position of node:)
5: Exit the program:
:Enter Choice:3
Enter Student name or id:112233
Name: Ali
ID: 112233
Department:  Ai engineering
Press any key to continue . . .
```

C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe

```
========(Main Menu)========
1: Add New Student:
2: Display Students:
3: Search Student:
4: Delete Student(Give position of node:)
5: Exit the program:
:Enter Choice:4
Total nodes: 2
Enter position: 1
Head deleted Successfully:
Press any key to continue . . .
```

Task2:

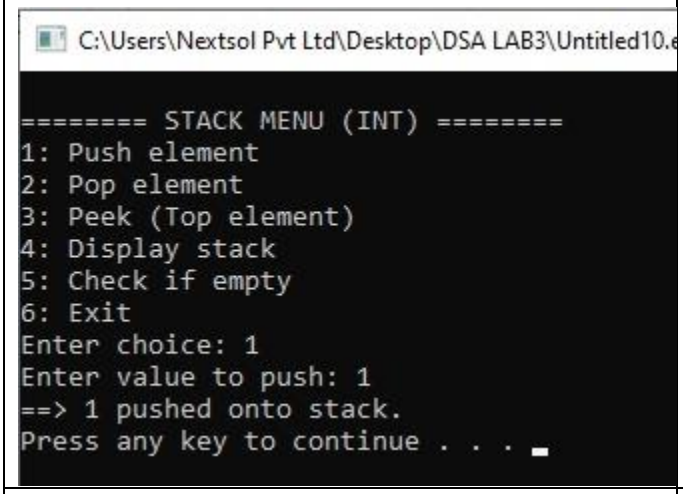
Stack using linklist.

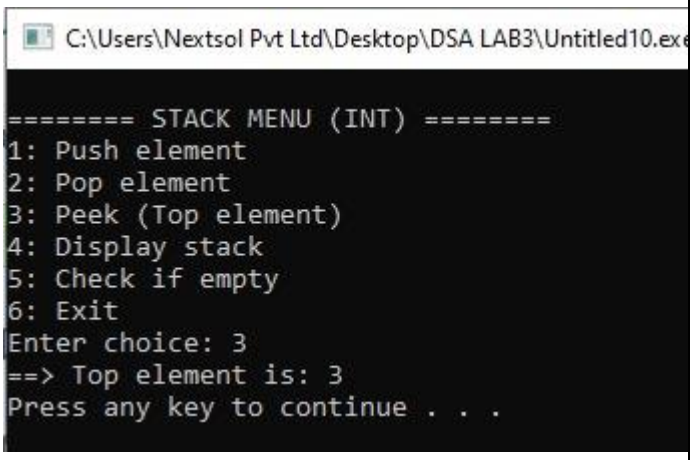
```cpp
#include<iostream>
#include<string>
using namespace std;
template<typename T>
class Node{
public:
T data;
Node*next;
Node(T val){
data=val;
next=NULL;}};
template<typename T>
class StackList{
private:
Node<T>*top;
public:
StackList(){
top=NULL;}
//insert
void push(T value){
Node<T>*newNode=new Node<T>(value);
newNode->next=top;
top=newNode;
cout<<"==> "<<value<<" pushed onto stack.\n";}
//Delete
void pop(){
if(isEmpty()){
cout<<"==> Stack Underflow! Nothing to pop.\n";
return;}
cout<<"==> "<<top->data<<" popped from stack.\n";
Node<T>*temp=top;
top=top->next;
delete temp;}
//empty check;
void peek(){
if(isEmpty()){
cout<<"==> Stack is empty.\n";
return;}
cout<<"==> Top element is: "<<top->data<<endl;}
bool isEmpty(){
return top==NULL;}
//display
void display(){
if(isEmpty()){
cout<<"==> Stack is empty.\n";
return;}
cout<<"\n--- Stack elements (TOP to BOTTOM) ---\n";
Node<T>*temp=top;
while(temp!=NULL){
cout<<"| "<<temp->data<<" |\n";
temp=temp->next;}
cout<<"----------\n";}
~StackList(){
while(!isEmpty()){
Node<T>*temp=top;
top=top->next;
delete temp;}}};
//main
int main(){
StackList<int>s;
int value;
while(1){
system("cls");
cout<<"\n======== STACK MENU (INT) ========\n";
```

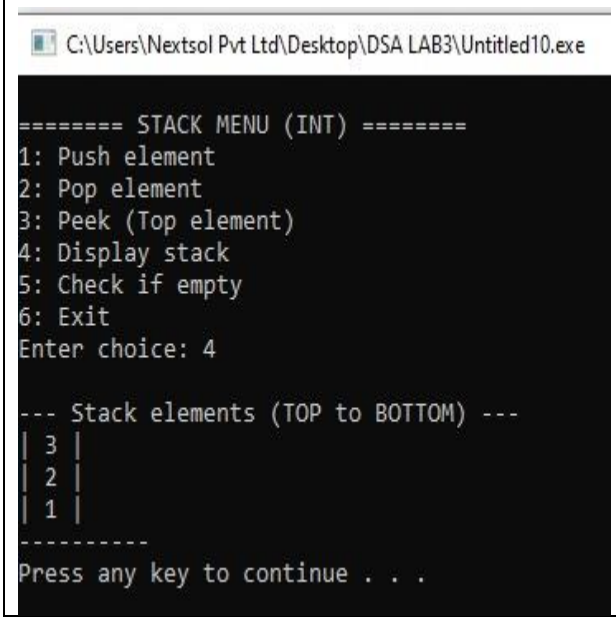
```
65  cout<<"1: Push element\n";
66  cout<<"2: Pop element\n";
67  cout<<"3: Peek (Top element)\n";
68  cout<<"4: Display stack\n";
69  cout<<"5: Check if empty\n";
70  cout<<"6: Exit\n";
71  cout<<"Enter choice: ";
72  int c;
73  cout<"Enter chocie:";cin>>c;
74  switch(c){
75  case 1:
76  cout<<"Enter value to push: ";
77  if(cin>>value){
78  s.push(value);
79  }else{
80  cout<<"Invalid value entered.\n";
81  cin.clear();}
82  system("pause");
83  break;
84  case 2:
85  s.pop();
86  system("pause");
87  break;
88  case 3:
89  s.peek();
90  system("pause");
91  break;
92  case 4:
93  s.display();
94  system("pause");
95  break;
96  case 5:
```

```
97  if(s.isEmpty()){
98  cout<<"==> The stack IS empty.\n";
99  }else{
100 cout<<"==> The stack IS NOT empty.\n";}
101 system("pause");
102 break;
103 case 6:
104 cout<<"Exiting program. Stack memory is being cleared...\n";
105 exit(1);
106 break;
107 default:
108 cout<<"Invalid choice. Please try again.\n";
109 system("pause");
110 break;}}
111 return 0;}
```

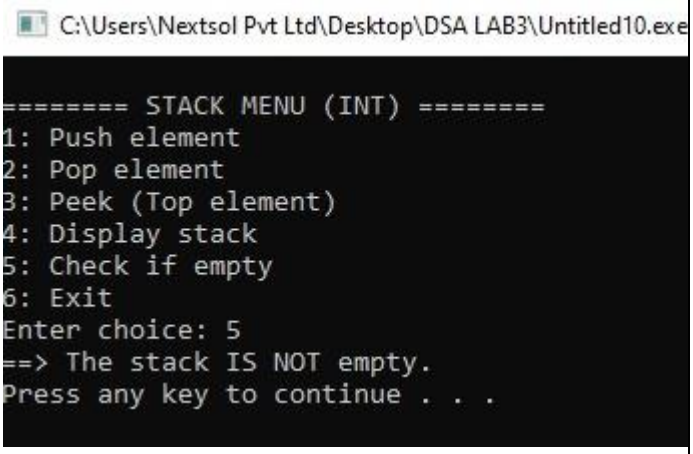
OUTPUT


C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.e
======== STACK MENU (INT) ========
1: Push element
2: Pop element
3: Peek (Top element)
4: Display stack
5: Check if empty
6: Exit
Enter choice: 1
Enter value to push: 1
==> 1 pushed onto stack.
Press any key to continue . . .


C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe
======== STACK MENU (INT) ========
1: Push element
2: Pop element
3: Peek (Top element)
4: Display stack
5: Check if empty
6: Exit
Enter choice: 3
==> Top element is: 3
Press any key to continue . . .


C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe
======== STACK MENU (INT) ========
1: Push element
2: Pop element
3: Peek (Top element)
4: Display stack
5: Check if empty
6: Exit
Enter choice: 4
--- Stack elements (TOP to BOTTOM) ---
| 3 |
| 2 |
| 1 |

Press any key to continue . . .


C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\Untitled10.exe
======== STACK MENU (INT) ========
1: Push element
2: Pop element
3: Peek (Top element)
4: Display stack
5: Check if empty
6: Exit
Enter choice: 5
==> The stack IS NOT empty.
Press any key to continue . . .

Conclusion:

- These two structures show the **flexibility** of using simple node connections.
- The **doubly circular list** offers continuous, two-way movement for endless loops or round-robin tasks.
- The **stack** strictly enforces the **LIFO** (Last-In, First-Out) rule on a linked list.
- This means all changes only happen at one end, which is perfect for managing temporary data.
- While the circular list emphasizes **bi-directional freedom**, the stack demands **strict order**.
- They are both essential but serve fundamentally **different design purposes**.
- Choosing the right one is key to building an **efficient** and functioning program.